

ECE 1508: Reinforcement Learning

Chapter 1: Introduction

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Fall 2026

Defining RL Problem

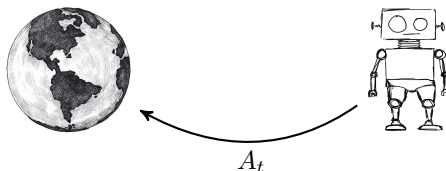
Let's make an agreement

$$\text{Reinforcement Learning} \equiv \text{RL}$$

In each RL problem, we have three main components

- ① *Agent* who is the one who *acts*
- ② *Environment* which responds to the *agent's actions*
- ③ *Interaction* means which communicate *actions* and *responds*

Problem Formulation: Action

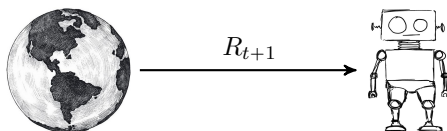


At time t , *agents* selects an *action* from a set of possible actions

$$A_t \in \mathbb{A} = \{a^1, \dots, a^M\}$$

$\mathbb{A}$ is *not* necessarily *discrete*, but we may assume so for now

Problem Formulation: *Reward*



Once *acted*, the *agent* gets some *reward* from the environment

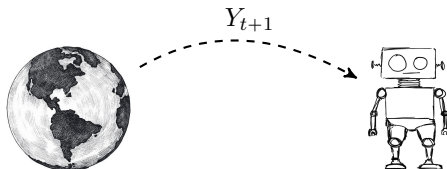
$$R_{t+1} \in \{r^1, \dots, r^L\}$$

We index by $t + 1$, as agent *gets the reward in the next time step*

↳ Again we assume a *discrete* set of possible rewards *only for simplicity!*

↳ If agent receives *no* reward, we could set $R_{t+1} = 0$

Problem Formulation: *Observation*

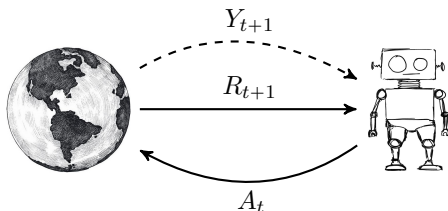


Before the next action, the *agent* also *observes* the *environment*

$$Y_{t+1} \in \mathbb{Y}$$

We index by $t + 1$, as agent *observes after each action*

Problem Formulation: *Trajectory*



Let's make an agreement¹

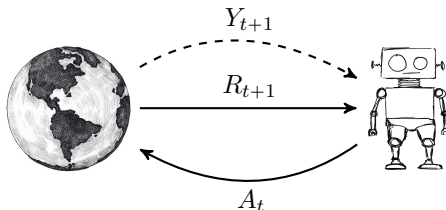
- ↳ At time $t = 1$ agent observes initial observation Y_1 and reward R_1
 - ↳ It then decides for action A_1
- ↳ As a consequence it gets observation Y_2 and reward R_2
 - ↳ It then decides for action A_2

...

This behavior continues for ever, i.e., $t = 1, \dots, \infty$

¹We keep it as in Sutton's book, as it's a common notation

Problem Formulation: *History*

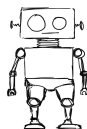


*History contains all happened up to time t , before **agent** acts in time t*

$$H_t = A_0, R_1, Y_1, A_1, R_2, Y_2, A_2, \dots, R_t, Y_t$$

*This is the information that has been collected by the **agent** up to time t*

Problem Formulation: *Final Goal of Agent*



The final goal of the **agent** is to

maximize what it is going to collect as **reward** in **future**

Future Return

At time t , the *future return* of the **agent** is

$$G_t = \sum_{i=0}^{\infty} \gamma^i R_{t+i+1}$$

for some constant $0 \leq \gamma \leq 1$ referred to as **discount factor**

- + What exactly is this *return*? Why do we have a **discount factor**?
- Let's open it up!

Problem Formulation: *Future Return*

The *future return* at time t looks like

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

We can look at it this way: after we decide to act A_t at time t

- We get *immediate* reward R_{t+1} in next time step
- This will also *impact our next action* A_{t+1}
 - ↳ after this we will get R_{t+2}
 - ↳ after the next action we will get R_{t+3}
 - ...
- We rather prefer to get the reward *sooner* than *later*!
 - ↳ large R_{t+1} is preferred to large R_{t+2}
 - ↳ large R_{t+2} is preferred to large R_{t+3}
 - ...
- We scale down gained reward with *discount factor* after each time step

Problem Formulation: *Future Return*

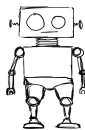
The *future return* at time t looks like

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Extreme choices of *discount factor* γ are *zero* and *one*:

- *Shortsighted* agent sets $\gamma = 0$
 - ↳ It only cares about the *immediate* reward
- *Forethoughtful* agent that sets $\gamma = 1$
 - ↳ It cares about *future* as much as it cares about now
 - ↳ It would accept less reward now, if it expects *larger compensation* in future

A Stochastic Framework



Agent looks at the **environment** as a **random object**

- Depending on agent's **action**, it returns a random **reward** and **observation**

Agent may also act **randomly**

- It is due to the **randomness** of **environment**

We assume that **environment** is a **state-dependent** system

Modeling Components: State

State-dependent Environment

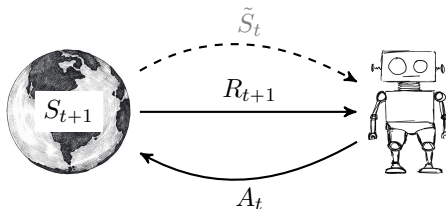
The environment at each time step has a particular state

$$S_t \in \mathcal{S} = \{s^1, \dots, s^N\} \leftarrow \text{set of possible states}$$

which along with the action A_t specifies its next state the reward it returns

- + Is there necessary a **discrete** set of possible states?
- **Not** necessarily! But we keep assuming whenever needed
- + How does this model impact the RL setting?!
- Let's see

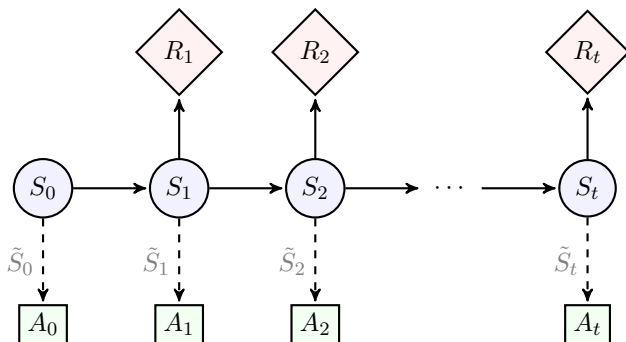
RL Framework: State-Representation



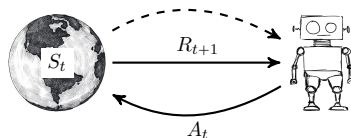
- ① Agent observes $\tilde{S}_t$
 - ↳ In **ideal world**, it can see the true state: $\tilde{S}_t = S_t$
 - ↳ In **real world**, it might see an **incomplete** version: $\tilde{S}_t \neq S_t$
- ② Based in the observed **state** $\tilde{S}_t$, agents decides to **act** A_t
- ③ **Environment** receives action A_t
 - ↳ It **rewards** the agent as R_{t+1}
 - ↳ It changes its **state** to S_{t+1}

RL Framework: *State-Representation*

So, we can plot the trajectory as a causal graph



RL Framework: Markovity of State



The **complete state** should give **all information** about the environment at each time: this means that at time t

the **reward of next time** R_{t+1} and the **next state** S_t

should be described completely by the **action** A_t and current **state** S_t

- + Can we guarantee that we always find such **state**?
- Well, we can assume that it **exists**, but maybe we do not have **access** to it

RL Framework: Markovity of State

We assume that *the state* S_t contains *all information* up to time t

↳ This state describes a *Markov process*

Markov Process

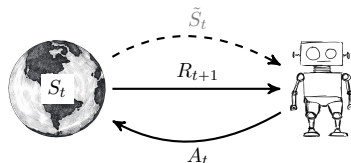
Sequence $S_1 \rightarrow S_2 \rightarrow \dots$ describe a Markov process if

$$\Pr \{S_{t+1} = s_{t+1} | S_t = s_t, \dots, S_1 = s_1\} = \Pr \{S_{t+1} = s_{t+1} | S_t = s_t\}$$

In other words: we assume that

whatever *we need to know about past* is always in the *last state*

RL Framework: Agent's Observation

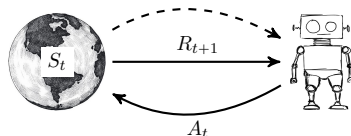


The agent could have different levels of access to the state

- **Full** observation of the state: $\tilde{S}_t = S_t$
 - ↳ This is an **optimal case**, but does not happen always
- **Partial** observation of the state: $\tilde{S}_t \neq S_t$
 - ↳ This is more **realistic**, as some agents are **restricted** in their observation

We consider **full** observation for the moment

RL Framework: *Environment's Behavior*



- ① *Rewarding function* that maps *state* S_t and *action* A_t to reward R_{t+1}

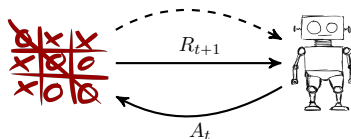
$$\mathcal{R}(\cdot) : \mathcal{S} \times \mathcal{A} \mapsto \{r^1, \dots, r^L\}$$

- ② *Transition function* that maps *state* S_t and *action* A_t to the next state S_{t+1}

$$\mathcal{P}(\cdot) : \mathcal{S} \times \mathcal{A} \mapsto \mathcal{S}$$

These mappings are in general *random*

Example: Tic-Tac-Toe



Let's consider the game of Tic-Tac-Toe: *in this game,*

- **Agent** is one of the players
 - ↳ **Action** is the move done by the player
- **Environment** is the **opponent and the game rules**
 - ↳ **State** in each time is the **status of the board**

Let's make it mathematically consistent!

Example: Tic-Tac-Toe

0	1	2
3	4	5
6	7	8

O		X
X	X	O
O		

$$S_t = \{0, 2, 3, 4, 5\}$$

$$A_t = 6$$

We can number each cell on the board; in this case,

- *Played cells* can be considered the state
 ↳ *State* in each time is the *status of the board*
- *Next cell played* is the action

- + Is it consistent with the Markovity assumption?
- Sure! Let's see

Example: Tic-Tac-Toe

O		X
X	X	O
O	X	

$$S_t = \{0, 2, 3, 4, 5\} \rightsquigarrow S_{t+1} = \{0, 2, 3, 4, 5, \textcolor{green}{6}, \textcolor{blue}{7}\}$$

$$\textcolor{red}{A}_t = 6$$

The player decides **only** based on the **status of the board**

↳ Its **action** at time t relies on **state** S_t

Next state also depends only on the **current status** and **player's move**

↳ S_{t+1} is **completely** described by **state** S_t and **action** A_t

$$\textcolor{green}{S}_{t+1} = S_t \cup \{\textcolor{red}{A}_t, \textcolor{blue}{O}_{t+1}\}$$

with $\textcolor{blue}{O}_{t+1}$ being **opponent's move**

Example: Tic-Tac-Toe

O		X
X	X	O
O		

$$S_t = \{0, 2, 3, 4, 5\}$$

$$A_t = 6$$

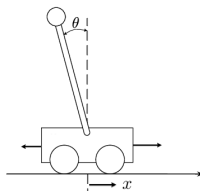
Transition function is probabilistic in this case

$$\mathcal{P}(S_t, A_t) = \begin{cases} \{0, 2, 3, 4, 5, 6, 1\} & \text{with probability } \Pr\{O_{t+1} = 1\} \\ \{0, 2, 3, 4, 5, 6, 7\} & \text{with probability } \Pr\{O_{t+1} = 7\} \\ \{0, 2, 3, 4, 5, 6, 8\} & \text{with probability } \Pr\{O_{t+1} = 8\} \end{cases}$$

Rewarding function is deterministic

$$\mathcal{R}(S_t, A_t) = \begin{cases} +1 & \text{if player wins after playing } A_t \\ -1 & \text{if player loses after playing } A_t \\ 0 & \text{if players draw or the game is not over} \end{cases}$$

Example: Cart-Pole Problem



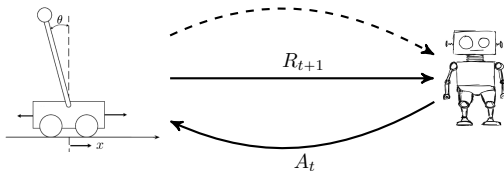
A pendulum is located vertically on a cart; at each time step, *player* can

- either shift the cart to the *right*
- or shift it to the *left*

The player *gets \$1* for each time step that

- the pendulum remains in an angle *less than θ* from the vertical line, and
- the cart is displaced with distance *less than x*

Example: Cart-Pole Problem



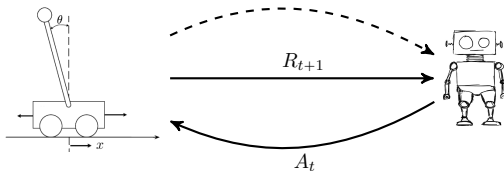
Agent is the player

- At time t , it acts as $A_t = \pm 1$, i.e., right or left

Environment is the cart-pole, game rule and the physical laws

- **State** at time t is the collection of physical parameters
 - ↳ Distance of cart, its velocity, angle of pendulum, angular velocity, . . .
 - ↳ **Current state** and **action completely** describe **next status** of pendulum
- It returns **reward** $R_{t+1} = 1$, if game is not over after **action** A_t

Example: Cart-Pole Problem



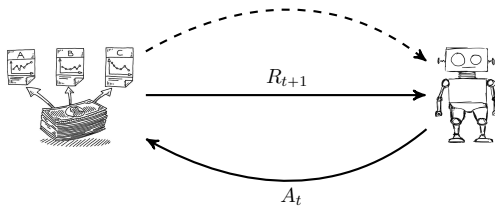
Transition function is completely described by dynamics of the system

$$\mathcal{P}(S_t, A_t) = \text{Solution to dynamic equations}$$

Rewarding function is further specified by the game rules

$$\mathcal{R}(S_t, A_t) = \begin{cases} +1 & \text{if game is not over after playing } A_t \\ 0 & \text{otherwise} \end{cases}$$

Example: Trading



Agent is the one who invests

- At time t , it acts by **either buying or selling**

Environment is the **market, investing portfolio, and transaction rules**

- **State** is a collection of **market and portfolio features**
- **Reward** R_{t+1} describes the profit made in each time frame

Obviously in this case, **transition function** is **not clear** to us!

Frozen Lake Game



The guy needs to get to the treasure

- + It could walk over frozen cells*
- If it runs to a shallow cell it falls into water*

If he decides for a direction, it could

- ✓ move to direction with probability $1 - p$*
- ✗ slip with probability p to **another** direction*

*The guy wants to learn the **best way**, i.e.,*

with minimal chance of failing

Playing in RL Framework

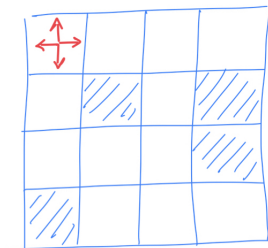
State $\equiv$ Location of the guy on the game board

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

$$\mathcal{S} = \{0, 1, \dots, 15\}$$

Playing in RL Framework

Action $\equiv$ Possible moves on the board



We can represent them with some integer

$$\mathbb{A} = \{0 \equiv \text{left}, 1 \equiv \text{down}, 2 \equiv \text{right}, 3 \equiv \text{up}\}$$

Playing in RL Framework

Reward $\equiv$ Possible outcomes of the game

	-1		-1
			-1
-1			+1

We have three possible cases

$$R \in \{0 \equiv \text{ongoing}, 1 \equiv \text{winning}, -1 \equiv \text{losing}\}$$

Playing in RL Framework

Rewarding Function

Reward of each *state-action* pair is a random variable

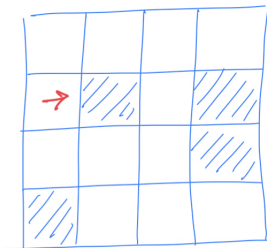
$$\mathcal{R}(s, a) = \begin{cases} \text{intended reward} & 1 - p \\ \text{reward of slipping direction} & p \end{cases}$$

Transition Function

Transition of each *state-action* pair is also a random variable

$$\mathcal{P}(s, a) = \begin{cases} \text{intended state} & 1 - p \\ \text{state of slipping direction} & p \end{cases}$$

Playing in RL Framework



For example, in the above image we could say

$$\mathcal{P}(\textcolor{green}{4}, \textcolor{red}{2}) = \begin{cases} 5 & 1-p \\ 0 \text{ or } 8 & p \end{cases} \quad \text{and} \quad \mathcal{R}(\textcolor{green}{4}, \textcolor{red}{2}) = \begin{cases} -1 & 1-p \\ 0 & p \end{cases}$$

Gymnasium



*Gymnasium is an API with some pre-implemented **environments***

- *We can call environments and access the **state** and **reward***
- *We can interact through **actions***
- *This is a powerful toolkit to test our RL algorithms*
 - ↳ *At some point, we may prefer implementing our own environment*

Gymnasium: Trying Frozen Lake Game

We can call the frozen lake environment easily in Gymnasium

```
1  import gymnasium as gym
2
3  def test():
4      # Create FrozenLake instance
5      env = gym.make('FrozenLake-v1', map_name="4x4", is_slippery=False, render_mode='human')
6
7      env.reset() # Initialize to state 0
8      terminated = False # True when agent falls in hole or reached goal
9      truncated = False # True when agent takes more than 200 actions
10
11     while(not terminated and not truncated):
12
13         action = env.action_space.sample()
14
15         # Execute action
16         state, reward, terminated, truncated, _ = env.step(action)
17         print(reward)
18         print(state)
19     env.close()
```

We will play around with it in Assignment 1